Especificación de Requerimientos y Diseño Técnico (ERS / RFC)

Tarifas Dinámicas y Ajuste de Liquidación de Órdenes de Servicio

- **Proyecto:** Bull Dog Solution
- **Documento:** Especificación técnica y de requerimientos
- **Fecha original:** 2026-09-28
- **Revisión de estado:** 2026-10-02
- **Estado:** Implementado y verificado
- **Autor:** Equipo de Arquitectura de Software

---

1. Contexto y Justificación del Negocio

En la operación de campo actual, los costos que cobran los técnicos varían frecuentemente en función de:

- El día en que se ejecuta la orden (fines de semana, días festivos, urgencias).
- Negociaciones puntuales en el sitio de trabajo o condiciones logísticas extraordinarias.
- Desviaciones detectadas con posterioridad a la finalización del servicio que requieren corrección contable antes de liquidar el pago.

Actualmente, las tarifas se encuentran definidas en la hoja de cálculo de Google Sheets (`DIRECTORIO TECNICO`), las cuales se sincronizan hacia la base de datos en la tabla `Technician` . Sin embargo, al completar una orden, el sistema toma de forma rígida esas tarifas maestras del técnico.

Principio Rector

Las tarifas del directorio en Google Sheets se conservan como valores base/por defecto. No se eliminan ni se altera su sincronización. No obstante, el sistema debe proveer la capacidad de sobreescribir las tarifas para una orden específica, tanto al inicio de su ejecución como tras haber sido finalizada.

---

2. Alcance y Reglas de Negocio

2.1 Control de Acceso y Roles (MANDATORIO)

- **Técnicos (`TECHNICIAN`):**

    - **No tienen permiso de modificar tarifas ni viáticos.**
    - Su interfaz móvil de ejecución (`OrderExecutionView.vue`) mantiene el flujo limpio habitual. No se añaden pasos bloqueantes ni campos de precio a su pantalla.

- **Administradores (`ADMIN`):**
    - Poseen control total sobre las tarifas y liquidación (`orders:manage`).
    - Pueden definir o modificar tarifas acordadas mientras la orden no esté finalizada ni cancelada (incluye `DRAFT` , `ASSIGNED` , `IN_PROGRESS` y órdenes devueltas a `REQUIRES_CORRECTION`).
    - Pueden ajustar precios y viáticos en órdenes ya finalizadas (`COMPLETED` , `FAILED_APPOINTMENT`).

2.2 Ciclo de Vida de los Precios en la Orden

1. **Valores por defecto:** Al asignar un técnico a una orden, las tarifas base del servicio y viático corresponden a las configuradas en su registro.
2. **Definición previa o al inicio:** Un administrador puede registrar tarifas personalizadas (`pricingOverrides`) para esa orden específica. Si un servicio no tiene tarifa personalizada, hereda la tarifa base del técnico.
3. **Cálculo al finalizar (`completeOrder` / `recordFailedAppointment`):** El motor de liquidación toma prioritariamente las tarifas de `pricingOverrides` si existen; en su defecto, utiliza las tarifas maestras del técnico.
4. **Ajuste post-cierre:** Si una orden ya está `COMPLETED` o `FAILED_APPOINTMENT` , el administrador puede editar directamente los precios unitarios o viáticos. Esto genera una nueva versión de precios (`pricingVersion`), recalcula los subtotales/totales, actualiza el snapshot y encola la actualización automática hacia Google Sheets.

---

3. Arquitectura y Principios de Diseño

Para evitar degradar la mantenibilidad del proyecto y no generar archivos monolíticos ("archivotes con muchas dependencias"), se adoptan los siguientes principios de arquitectura limpia:

1. **Responsabilidad Única (SRP):**
    - `pricing.ts` : Función pura de cálculo sin dependencias de base de datos ni I/O.
    - `OrderPricingService` : Nuevo servicio desacoplado encargado exclusivamente de la gestión, validación, actualización y auditoría de tarifas de órdenes.
    - `orders.service.ts` y `order-finalization.service.ts` : No se sobrecargan de lógica contable accesoria.
2. **Auditoría e Inmutabilidad de Versiones:**
    - Todo cambio de tarifas registra un evento en la tabla `AuditLog` (`ORDER_PRICING_CONFIGURED` o `ORDER_PRICING_ADJUSTED`).
    - Cada ajuste post-cierre incrementa `pricingVersion` , preservando la trazabilidad.

3. **Sincronización Idempotente con Google Sheets:**
    - La sincronización del reporte técnico (`SHEET_TECHNICAL_REPORT`) utiliza `idempotencyKey: REPORT:${orderId}:${pricingVersion}` . Cualquier ajuste sustituye las operaciones pendientes (`SUPERSEDED`) y programa la escritura de la nueva liquidación en la hoja de cálculo.

---

4. Especificación Técnica de Componentes

4.1 Modelo de Datos (Prisma)

Se agrega el campo `pricingOverrides` en el modelo `Order` de `prisma/schema.prisma` :

```
model Order {
  // ... campos existentes ...
  pricingUnitRate          Decimal?          @db.Decimal(10, 2)
  pricingAppliedQuantity   Int?
  pricingBaseCost          Decimal?          @db.Decimal(10, 2)
  pricingDepartureCost     Decimal?          @db.Decimal(10, 2)
  pricingTravelExpenses    Decimal?          @db.Decimal(10, 2)
  pricingTotal             Decimal?          @db.Decimal(10, 2)
  pricingCurrency          String?           @default("MXN")
  pricingSnapshotAt        DateTime?         @db.Timestamptz(3)
  pricingVersion           Int?              @default(1)
  pricingBreakdown         Json?             @db.JsonB
  pricingOverrides         Json?             @db.JsonB   // <--- NUEVO CAMPO
  // ... relaciones y auditoría ...
}
```

Estructura de `pricingOverrides` (`Json`):

```
{
  "rates": {
    "INSTALLATION": "650.00",
    "REVIEW": "300.00"
  },
  "travelExpenses": "400.00",
  "reason": "Tarifa convenida por día festivo"
}
```

4.2 Estrategia de Tipado y Precisión Financiera (Type Safety & Precision)

Para prevenir discrepancias contables y errores de redondeo por punto flotante (IEEE-754 en JavaScript, ej. `0.1 + 0.2 = 0.30000000000000004`), el manejo de tipos se estructura estrictamente en 4 capas:

1. **Transporte y Contrato (`@puk/contracts`):**
   - **Fuente de verdad única (SSOT)** definida con Valibot.
   - Todo importe monetario viaja como `string` decimal formateado con 2 dígitos decimales (`/^\d+(\.\d{1,2})?$/`). Cero tipos `number` en JSON de transporte para precios.
   - Las claves de servicios usan el enum fuertemente tipado `OrderServiceType` .
2. **Dominio y Cálculo (`apps/api` - `pricing.ts`):**
   - Los strings se convierten a instancias de `Prisma.Decimal` (basado en `decimal.js`).
   - Toda la aritmética (`plus` , `times`) opera en precisión arbitraria con redondeo explícito (`ROUND_HALF_UP` a 2 decimales).
   - Los snapshots se generan llamando a `.toFixed(2)` para reingresar al contrato como string inmutable.
3. **Persistencia (PostgreSQL / Prisma):**
   - `Order.pricingOverrides` : Almacenado como `Json?` (`@db.JsonB`), validado en runtime mediante el schema de Valibot al deserializar.
   - Columnas monetarias calculadas (`pricingTotal` , `pricingBaseCost` , etc.): Preservadas como `Decimal(10, 2)` .
4. **Presentación e Interfaz (`apps/web`):**
   - Componentes de formulario de Naive UI (`NInputNumber`) manipulan temporalmente valores numéricos en el estado local reactivo.
   - La capa de cliente API (`api.ts`) es responsable de transformar `number | null` a `string` (`val.toFixed(2)`) antes de emitir la petición HTTP, garantizando coherencia con el contrato.

Esquemas Valibot en `@puk/contracts` (`packages/contracts/src/orders.ts`):

```
export const PricingRateOverrideSchema = v.pipe(
  v.string(),
  v.regex(/^\d+(\.\d{1,2})?$/, 'Debe ser un importe válido (ej. 450.00)'),
);

export const OrderPricingOverridesSchema = v.object({
  rates: v.optional(v.record(OrderServiceTypeSchema, PricingRateOverrideSchema)),
  travelExpenses: v.optional(v.nullable(PricingRateOverrideSchema)),
  reason: v.optional(v.nullable(v.pipe(v.string(), v.maxLength(500)))),
});

export type OrderPricingOverrides = v.InferOutput<typeof OrderPricingOverridesSchema>;
```

4.3 Motor de Cálculo Puro (`pricing.ts`)

Se extiende la entrada de cálculo para admitir tarifas personalizadas:

```
export interface CalculateOrderPricingInput {
  readonly technician: TechnicianPricingSource;
  readonly serviceType?: OrderServiceType | null;
  readonly equipmentCount?: number;
  readonly isHomeService: boolean;
  readonly pricingVersion?: number;
  readonly snapshotAt?: Date;
  readonly items?: ReadonlyArray<{
    serviceType: OrderServiceType;
    equipmentCount?: number;
  }>;
  readonly overrides?: OrderPricingOverrides | null; // <--- NUEVO
}
```

- En la resolución de tarifa (`resolveServiceRate` o interna): si `overrides?.rates?.[serviceType]` existe, se utiliza ese valor y se omite la tarifa del técnico.
- En viáticos: si `overrides?.travelExpenses` está provisto, se aplica directamente dicho importe.

4.4 Servicio Dedicado: `OrderPricingService`

Ubicación: `apps/api/src/modules/orders/order-pricing.service.ts`

Responsabilidades:

1. `setOrderPricingOverrides(actorUserId: string, orderId: string, input: OrderPricingOverrides)` :
    - Permite a administradores configurar o actualizar tarifas pactadas en órdenes que no estén finalizadas ni canceladas; la vista ofrece el control a administradores en cualquier estado excepto `CANCELLED` .
    - Persiste en `order.pricingOverrides` .
    - Registra auditoría `ORDER_PRICING_CONFIGURED` .
2. `adjustFinalizedOrderPricing(actorUserId: string, orderId: string, input: OrderPricingOverrides)` :
    - Valida que la orden esté en `COMPLETED` o `FAILED_APPOINTMENT` .
    - Carga la orden con sus unidades, equipos y técnico.
    - Ejecuta `calculateOrderPricing` con los nuevos `overrides` .
    - Dentro de una transacción de dominio (`domainTransaction`):
        - Incrementa `pricingVersion` .
        - Actualiza `pricingBaseCost` , `pricingTravelExpenses` , `pricingTotal` , `pricingBreakdown` , `pricingSnapshotAt` y `pricingOverrides` .
        - Marca operaciones pendientes de `SHEET_TECHNICAL_REPORT` como `SUPERSEDED` .
        - Encola nueva operación `SHEET_TECHNICAL_REPORT` con `formatTechnicalReportRow` .
        - Registra auditoría `ORDER_PRICING_ADJUSTED` .

4.5 Endpoints HTTP (`apps/api`)

En `orders.router.ts` :

- `PATCH /api/orders/:id/pricing` :
  - Middleware: `authorize('orders:manage')` .
  - Validación del cuerpo con `OrderPricingOverridesSchema` .
  - Enruta según el estado de la orden a `setOrderPricingOverrides` o `adjustFinalizedOrderPricing` .

4.6 Frontend (`apps/web`)

1. **Componente Reutilizable (`OrderPricingModal.vue`):**
   - Modal de Naive UI (`NModal` , `NForm` , `NInputNumber`).
   - Lista cada tipo de servicio presente en la orden con su tarifa base actual y el campo para ingresar tarifa personalizada.
   - Campo para costo de viáticos / salida.
   - Campo opcional de justificación o nota del ajuste.
2. **Integración en `OrderDetailView.vue` (Vista Administrativa):**
   - En la sección "Desglose de Liquidación Técnica":
     - Si la orden está en estado `ASSIGNED` o `IN_PROGRESS` : botón "Configurar Tarifas Acordadas".
     - Si la orden está en estado `COMPLETED` o `FAILED_APPOINTMENT` : botón "Ajustar Precios de Liquidación".
   - Al guardar, refresca reactivamente la orden y muestra notificación de éxito.

4.7 Formateo Multilínea en Reporte Técnico de Google Sheets (`technical-report-formatter.ts`)

En órdenes con múltiples unidades y servicios multilínea (salvo citas fallidas):

- Las columnas F (`economico`), G (`serie`), H (`imeiId`), K (`servicio`) y L (`costo`) utilizan `\n` para alinear cada renglón por unidad en orden cronológico.
- La columna L (`costo`) resuelve el costo de cada unidad individual (`resolveUnitCost`) con base en el desglose congelado (`pricingBreakdown`), tarifas sobreescritas (`pricingOverrides`) o tarifa unitaria, multiplicado por el número de equipos de la unidad (o 1 para reemplazo de equipos).
- Las columnas M (`viaticos`) y N (`total`) conservan el valor global consolidado de la orden.

---

5. Matriz de Impacto y Mitigación de Riesgos

Componente	Nivel de Riesgo	Estrategia de Mitigación
Sincronización Google Sheets	Bajo	Se reutiliza el formateador `formatTechnicalReportRow` y el sistema de reintentos e idempotencia existente por versión.
Reportes PDF de Operación	Nulo	El reporte PDF no contiene datos financieros ni tarifas de técnicos; solo evidencias operativas.
Flujo de Técnicos en Campo	Nulo	No se altera la experiencia ni los endpoints utilizados por los técnicos en campo.
Integridad de Código Existente	Mínimo	No se modifica la firma base de funciones existentes; los overrides son estrictamente opcionales.

---

6. Fases de Implementación Propuestas

1. **Fase 1: Contratos y Persistencia**
   - Crear migración Prisma con la columna `pricingOverrides` en `Order`.
   - Definir esquemas Valibot en `packages/contracts`.
2. **Fase 2: Motor de Cálculo y Pruebas Unitarias**
   - Adaptar `calculateOrderPricing` en `pricing.ts` para consumir `overrides`.
   - Añadir batería de pruebas unitarias en `pricing.test.ts`.
3. **Fase 3: Servicio de Negocio y Endpoints**
   - Implementar `OrderPricingService` con auditoría y re-sincronización de Sheets.
   - Exponer endpoint `PATCH /api/orders/:id/pricing` protegido por `orders:manage`.
   - Añadir pruebas de integración en `orders.router.test.ts`.
4. **Fase 4: Frontend Administrativo**
   - Implementar `OrderPricingModal.vue`.
   - Integrar modal y botones de acción en `OrderDetailView.vue`.
5. **Fase 5: Verificación End-to-End**
   - Ejecutar suite de pruebas de regresión y prueba de flujo completo.
6. **Fase 6: Formateo Multilínea de Costos en Reporte Técnico**
   - Desglosar columna L (`costo`) renglón por renglón alineada con las unidades y servicios en Google Sheets.
   - Validar suite unitaria de formateo de reporte con servicios homogéneos y mixtos.